

ECE297 Quick Start Guide

Glade and GTK

“A user interface is like a joke. If you have to explain it, it’s not that good.”

–Martin LeBlanc

1 Overview

GTK is used to build most Linux GUI applications such as Chromium and Inkscape. GNOME (the Linux Desktop Environment you are using in the labs) is itself based on GTK. You can easily convert the UI of your mapper application to a layout that resembles your favourite GUI application with features such as a menu bar and popup menus, etc. EZGL is a wrapper around GTK that makes many operations easier to program, and it is strongly recommended to use EZGL for all of your drawing functions (lines, polygons, text, essentially drawing your map). However, EZGLs interactive user interface capabilities are limited. GTK will allow you to create more advanced User Interface widgets, like dropdown menus, overlays, and even search bars with autocomplete functionality.

GTK widgets can be created at run time with code, or loaded through a main.ui file. Main.ui files can be created with a program called Glade, which allows you to visually create your UI. It is strongly recommended to use Glade to extend your UI, if you do choose to go above and beyond the features EZGL offers.

1.1 Dependencies

Gtk depends on the GTK3 library. Glade and this library are both present on the UG machines. We are using version 3 of the gtk API (gtk3) and the C language bindings (which can be used in C++ code too); these C-style function calls are the most common way to use gtk.

See <https://gnome.pages.gitlab.gnome.org/gtk/gtk3/> for a complete list of the gtk widgets (UI elements) and functions available.

1.2 Opening Glade

On the *ugXXX* machines simply `cd` to the directory containing your mapper, then type:

```
glade libstreetmap/resources/main.ui
```

to open your main.ui file.



Moving or editing your main.ui file while running your mapper may cause errors.

2 Example Button Creation

There are four steps to creating a new UI element:

- Add it to the main.ui file through Glade (see above).
- Edit its position/size/initial properties using Glade.
- Connect it to its callback function.
- Write its callback function.

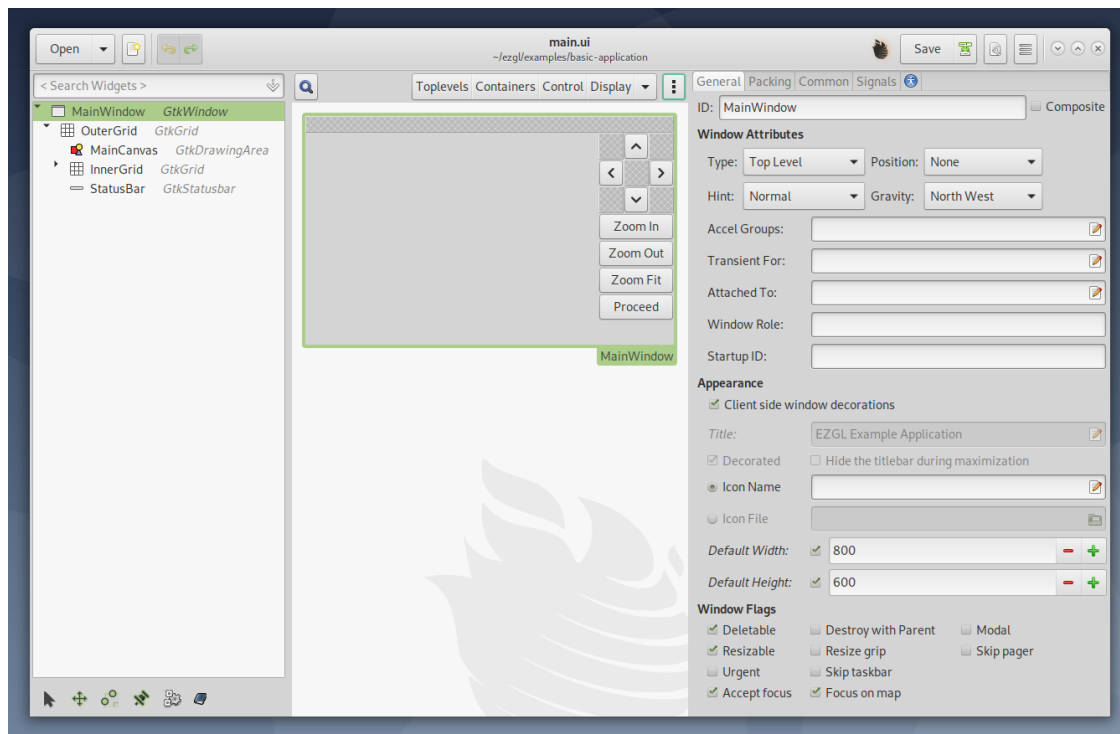


Figure 1: The Glade program.



This section will walk you through creating a UI element in Glade and adding functionality through code. We will be using the example program described in the EZGL Quickstart. Open `basic_application.cpp` in a code editor and `main.ui` with Glade.

2.1 Adding a new UI Element

This section will step you through changing the UI layout of the `basic_application.cpp` program described in the EZGL Quickstart.

Before starting, let's first go through the existing UI layout described in the `main.ui` file. The UI consists of a `GtkWindow` called `MainWindow`. A `GtkWindow` can only have one child, so we need to use a `GtkGrid` to manage multiple children. Hence the `MainWindow` has one child called `OuterGrid` which is a 2x2 `GtkGrid`. The top left slot of the grid is occupied by a `GtkDrawingArea` called `MainCanvas` which we use to draw the graphics. The top right slot is another `GtkGrid` called `InnerGrid` that contains the predefined buttons. A status bar is placed in the two bottom slots.

To change the UI layout, you can manually edit the `main.ui` file; however, it is recommended to use a UI designer program such as *Glade*.

Glade will display the current UI layout along with the widgets you can add as shown in Fig. 1. For this example, we will be adding a new element to our UI, a `GtkSwitch`. It is a switch with an on or off state, which is useful for toggling settings and visualization options (You can see its documentation here: <https://docs.gtk.org/gtk3/class.Switch.html>). To add a widget, you need to have somewhere to put it, so we will add another row to the `InnerGrid` widget. Click the `InnerGrid` widget on the left sidebar, which will open its specific properties in the left sidebar. Under the *General* tab, increase the rows count by 2. This will give us space for a new widget and its label.

Now go to the *Control* tab in the top toolbar, select a `GtkSwitch` widget, and drag it to the last row of your `InnerGrid`. Under the *General* tab, set its ID to "ExampleSwitch", and hit Active so it is on by default. Make sure your ID's are unique and logical, as you will be using them in your code to interact with your widgets. Now left-click anywhere on your window, and press the *Preview Snapshot* option to see a preview of your UI.

2.2 Editing Widget Size and Position

You should be able to see your switch underneath the Proceed button. However, you should also note that the Left Arrow is now larger and disproportionate relative to the other arrows. This is because a column in a grid will have its width set based on the width of its largest widget. In order to fix this, set your Switch *Width* to 3 in the *Packing* tab. Preview your UI. The arrows are now fixed, but our switch looks unnaturally large and extended.

The *Common* tab provides a large number of common options and settings for all widget types, including opacity, visibility, and spacing. You can use the *Expand* settings to set a widget to consume as much space as it can, and the *Alignment* settings to configure how a widget is placed in its space. Play around with the different options and preview your UI to figure them out. To fix our switch, set the Horizontal Alignment to Centre.

Now using what you've learned, add a `GtkLabel` in the empty row above your switch. Set the label to say "My Switch:".

Preview your ui to make sure it looks right, then SAVE and exit Glade. Your UI should look as shown in Fig. 2, with a new switch element at the bottom of the right menu.

2.3 Connect UI Elements To Code

Now that we have created our new button, we need to connect it using something called a callback function. A callback function is called when a `GtkWidget` sends a given signal, and a signal is an event created when a user interacts with a widget. To see the signals of a widget, you can go to the *Signals* tab of the right hand menu in Glade or to the documentation at [this link](#). To connect a signal to a callback function, you need to use the `g_signal_connect` function as listed below.

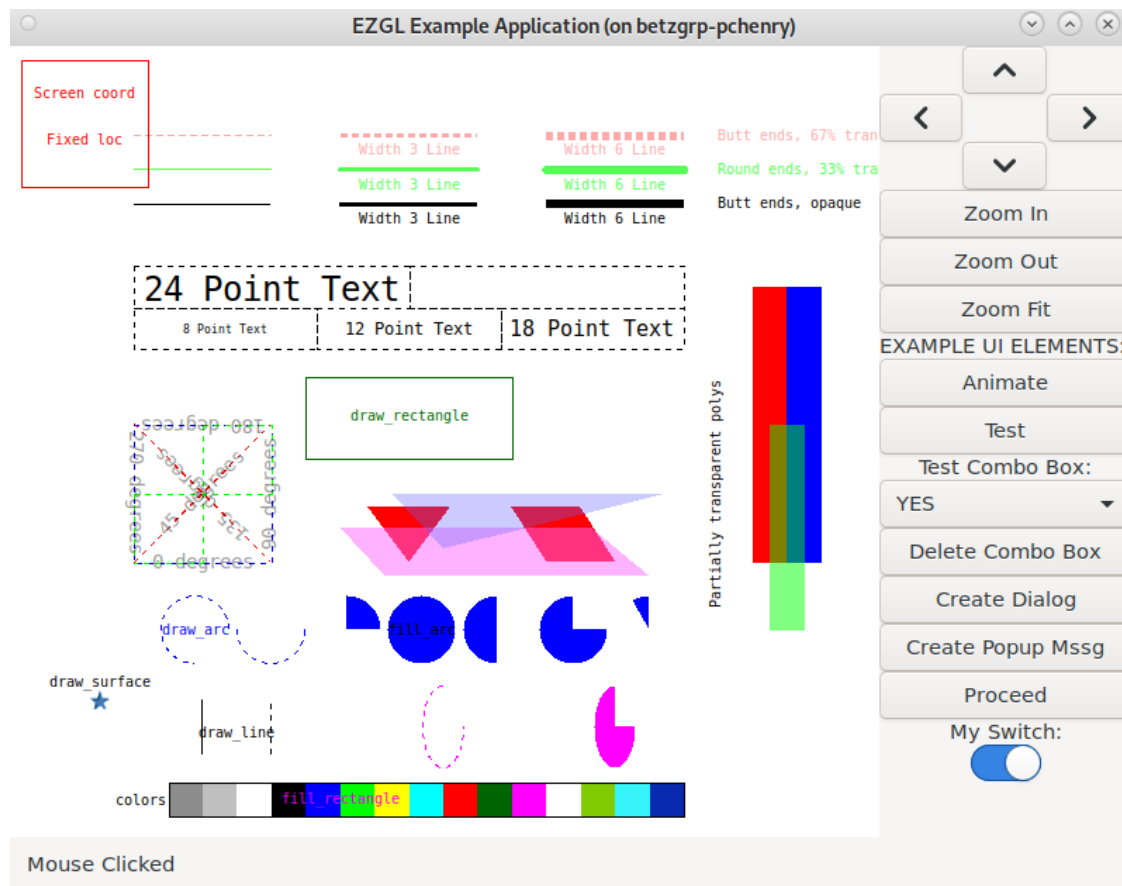


Figure 2: Example Basic Application with a GTKSwitch added.

```
// Get a pointer to the UI widget, using the ID string you set for it in glade
GObject *my_widget = application->get_object("ID I set in Glade");
g_signal_connect(
    my_widget,      // pointer to the UI widget
    "signal_name", // Signal name (from the widget documentation) e.g. "clicked"
    G_CALLBACK(my_func), // name of callback function (you write this function)
    NULL // pointer to (arbitrary) data to pass to callback my_func or NULL for no data
);
```

If you have added a UI element that should always be visible in the main window, this `g_signal_connect` call should be done in the `initial_setup` routine passed to `application.run`.

You'll find that you need an `ezgl::application*` for most of your callback functions, in which case the fourth arg should be an application ptr. If you need to pass more than just one data type, you can create a struct, initialize it with the values you need to pass, and set a ptr to the struct as your fourth argument (IT MUST BE A POINTER).

For our example switch, you'll want to add the following code to the example `initial_setup` function in `basic_application.cpp`:

```
// Getting a pointer to our example switch
```

```
GObject *example_switch = application->get_object("ExampleSwitch");
g_signal_connect(
    example_switch,      // pointer to the UI widget
    "state-set", // Signal state of switch being changed
    G_CALLBACK(example_cbk), // name of callback function (you write this function:
                           // make sure its declaration is visible)
    application // passing an application ptr.
);
```

Note you should declare *example_cbk* near the top of the file.

2.4 Writing Callback Functions

Now we need to write our callback function. Its important to be aware that different signals for different objects will have different declarations. To see the appropriate declaration, go to the docs for the class/widget you are using, scroll down to the signals, and click on the signal. For example, for a `GtkSwitch`, the signal *state-set* is emitted when the user changes the state of the switch. The name of the function does not matter, but the arguments should be the same. For example, for the *state-set* signal of a *GtkSwitch*, the docs say:



Figure 3: The documentation for the *state-set* signal of the `GtkSwitch`

User data is a void pointer, meaning it can point to any data type. You can declare it as a pointer to a set data type, i.e:

```
void example_cbk(GtkSwitch* , gboolean state, ezgl::application* app);
```

As long as you pass it a pointer of the stated type when you connect the signal, this will work. You can also keep it as a void pointer and cast it in your function.

Declare your *example_cbk* at the top of *basic_application.cpp*. To test our button, insert the following function:

```
//Note we comment out the variable name for the button. this will prevent the compiler
warning caused by an unused variable
void example_cbk(GtkSwitch* /*self*/, gboolean state, Eogl::application* app){
    if(state)
        app->update_message("Switch got turned on");
    else
        app->update_message("Switch got turned off");
}
```

Now compile and test your basic application. If the status bar shows the appropriate message, you have succeeded.

There are a lot of GTK Widgets that you can add to your application beyond Switches and Buttons. However, the general process is the same as shown above:

- Create a space for your item in Glade.
- Drag and drop the widget to that slot.
- Give the widget an ID.
- Connect the widget to a callback function with *g_signal_connect* in the *initial_setup* function.
- Use the [docs](#) to find the appropriate signal and cbk function format.
- Write your callback function.

Simply follow this method, and refer to the [docs](#)/[piazza](#)/[stack overflow](#) if you have questions/errors.

3 Creating a GtkEntry

One of the most useful widgets you can add to your User Interface is a GtkEntry, which is essentially a text box the user can type into.

You can find GtkEntry under the *Control* tab in Glade. Add a Gtk Entry to your UI and name it "TextEntry".

To access the text in a GtkEntry, we need a pointer to it. Then we can call the *gtk_entry_get_text* method function. To test this functionality, save your changes, then replace your *test_button_cbk* function with the following code:

```
/**
 * A callback function to test the Test button. Changes application message when button
 * is pressed
 */
void test_button_cbk(GtkWidget /*widget*/, Eogl::application *application)
{
    //Getting a pointer to our GtkEntry named "TextEntry"
    GtkEntry* text_entry = GTK_ENTRY(application->get_object("TextEntry"));

    //Getting text from our entry
}
```

```
const gchar* text = gtk_entry_get_text(text_entry);

//Updating the status bar message to be the text in our entry
application->update_message(text);

// Redraw the main canvas
application->refresh_drawing();
}
```

Test the functionality of your new entry. Hitting the Test button should now make the message be the text typed in your GtkEntry.

4 Casting

Gtk is a library based on inheritance, which allows child classes to use the methods of their parent's classes. All objects are derived from the *GObject* class, and usually the *GtkWidget* class as well. You will definitely run into scenarios where you need to use the method functions of multiple classes on the same object. To do this, you will need to cast pointers to the appropriate types, which essentially means changing the type. In GTK, this is done through macro functions that look like this:

```
//the get_object function returns a GObject type ptr
GObject* my_switch_ptr = application->get_object("ExampleSwitch");

//However, if we want to access the switch method functions, we need to cast it
bool switch_state = gtk_switch_get_active(GTK_SWITCH(my_switch_ptr));
//To cast to a Gtk type, simply write its name in full caps.

/* If we want to hide our switch, we need a GtkWidget*. Again, we cast it:
 * Note that hiding the switch means it disappears from the screen and everything adjusts
 * around it. It is not destroyed and
 * will return to normal with gtk_widget_show */
gtk_widget_hide(GTK_WIDGET(my_switch_ptr));
```

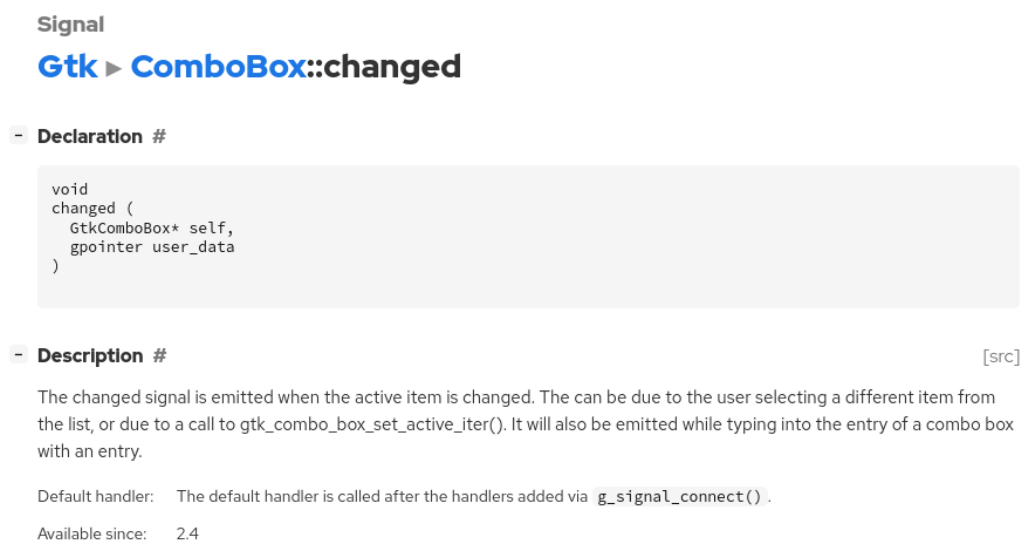
What this does under-the-hood is essentially tell the program that this is a pointer to an object of the given type, allowing it to access the member methods of that type. You can cast to any ancestor of the actual type (i.e a GtkSwitch can become a GtkWidget, but it can not become a GtkEntry). Gtk will throw a warning and null out the given pointer if you attempt an illegal cast. It will also throw warnings if you put in the wrong type of pointer into a function. The docs will list the ancestors and direct descendants of a given class.

5 Documentation

GTK is a massive library, with hundreds of objects and thousands of functions. Luckily, its very well documented and widely used, In addition to Piazza and StackOverflow, the [Gtk Docs](#) are an

invaluable source of information. Its a bit intimidating at first, so to help you out, here's a few general use cases:

Signals + Callback Functions When you are trying to write a callback function for a specific signal, go to the class page first, then scroll down to the *Signals* tab, click the signal you are searching for, and use the given callback. To give a concrete example, consider this [Example Signals Page](#) for a GTK ComboBox. A combo box is essentially a drop down menu with a set list of options. You can create simple versions with EZGL, but will need Glade/GTK to create Combo boxes with text entries. For this example, you can see it gives the function prototype for the callback function, as well as a description that notes when the signal will be created (Through both user and code changes to the selected option).



The screenshot shows the GTK documentation page for the `GtkComboBox::changed` signal. At the top, it says "Signal" and "Gtk ► ComboBox::changed". Below this is a section titled "Declaration #" which contains the following C code snippet:

```
void
changed (
    GtkComboBox* self,
    gpointer user_data
)
```

Below the declaration is a section titled "Description #" with a "[src]" link. The description states: "The changed signal is emitted when the active item is changed. The can be due to the user selecting a different item from the list, or due to a call to `gtk_combo_box_set_active_iter()`. It will also be emitted while typing into the entry of a combo box with an entry."

Below the description is a section titled "Default handler:" which states: "The default handler is called after the handlers added via `g_signal_connect()`."

At the bottom, it says "Available since: 2.4".

Figure 4: Documentation Page for the "Changed" signal of a GtkComboBox

Method Functions/Accessors To see a list of method functions for a given object, search the object in the sidebar, go to its page, and scroll to instance methods. These will generally be accessors + mutators, as well as some more unique functions. Each function will have its own page with usage tips/explanations. Example Page: https://docs.gtk.org/gtk3/method.ComboBox.get_active_id.html

Casting The documentation will list direct descendants and all direct ancestors of a given class. You can cast pointers to point to ancestor classes. If you need to cast to a child class, then you should ensure that the child class is the actual class of the object. Example Page: <https://docs.gtk.org/gtk4/class.ComboBox.html>